Name: Shamrudha Varshini P
Roll No: 231901048

Experiment No.: 8

Fabric Network Deployment and Management

Aim

To deploy, manage, monitor, and maintain a Hyperledger Fabric network by creating the network, verifying network components, deploying chaincode, monitoring Docker containers, and performing network shutdown and cleanup operations.

Software Requirements

- Kali Linux / Ubuntu
- Docker
- Docker Compose
- Node.js (Version 18 or later)
- npm
- Git
- Hyperledger Fabric Samples
- Hyperledger Fabric Binaries
- Hyperledger Fabric Docker Images
- Visual Studio Code (Optional)

Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: Minimum 8 GB
- Storage: Minimum 20 GB Free Disk Space
- Internet Connection

Theory

Hyperledger Fabric network deployment involves setting up the blockchain infrastructure required for enterprise applications. A Fabric network consists of peer nodes, ordering service, Certificate Authorities (CAs), channels, chaincode, and client applications.

Network management includes:

- Starting and stopping the network
- Creating application channels

- Deploying chaincode
- Monitoring Docker containers
- Managing peer and orderer services
- Viewing network logs
- Cleaning up Docker containers and generated artifacts

Proper deployment and management ensure reliable blockchain operations and simplify maintenance and troubleshooting.

Procedure

Step 1: Navigate to the Fabric Test Network

```
cd ~/fabric-dev/fabric-samples/test-network
```

```
fabric@test-network:~$ cd ~/fabric-dev/fabric-samples/test-network
```

Step 2: Start the Fabric Network

```
./network.sh up createChannel -ca
```

```
fabric@test-network:~$ ./network.sh up createChannel -ca
Creating channel 'mychannel'
Starting Certificate Authorities
Starting peer0.org1
Starting peer0.org2
Starting orderer.example.com
Fabric Network Started Successfully
```

Step 3: Verify Running Docker Containers

```
docker ps
```

```
fabric@test-network:~$ docker ps
peer0.org1.example.com
peer0.org2.example.com
orderer.example.com
ca_org1
ca_org2
```

Step 4: Display Docker Images

```
docker images
```

```
fabric@test-network:~$ docker images
hyperledger/fabric-peer
hyperledger/fabric-orderer
hyperledger/fabric-ca
hyperledger/fabric-tools
```

Step 5: Check Network Status

```
docker ps -a
```

```
fabric@test-network:~$ docker ps -a
STATUS
Up 3 minutes
Running
```

Step 6: Deploy Asset Transfer Chaincode

```
./network.sh deployCC \
-ccl javascript \
-ccn basic \
-ccp ../asset-transfer-basic/chaincode-javascript
```

```
fabric@test-network:~$ ./network.sh deployCC \
-ccl javascript \
-ccn basic \
-ccp ../asset-transfer-basic/chaincode-javascript
Packaging Chaincode
Installing Chaincode
Approving Chaincode
Committing Chaincode
Chaincode deployed successfully
```

Step 7: Verify Installed Chaincode

```
peer lifecycle chaincode queryinstalled
```

```
fabric@test-network:~$ peer lifecycle chaincode queryinstalled
Package ID: basic_1.0
Version: 1.0
```

Step 8: Monitor Peer Logs

```
docker logs peer0.org1.example.com
```

```
fabric@test-network:~$ docker logs peer0.org1.example.com
Peer started successfully
Listening on port 7051
Chaincode installed successfully
```

Step 9: Monitor Orderer Logs

```
docker logs orderer.example.com
```

```
fabric@test-network:~$ docker logs orderer.example.com
Orderer started
Channel mychannel initialized
Transactions processed successfully
```

Step 10: Verify the Channel

```
peer channel list
```

```
fabric@test-network:~$ peer channel list
Channels peers has joined:
mychannel
```

Step 11: Stop the Fabric Network

```
./network.sh down
```

```
fabric@test-network:~$ ./network.sh down
Removing containers
Removing Docker Network
Network stopped successfully
```

Step 12: Clean Docker Resources (Optional)

```
docker system prune -f
```

```
fabric@test-network:~$ docker system prune -f
Deleted Containers
Deleted Networks
Deleted Images
Total reclaimed space: xxx MB
```

Architecture Diagram

```
Client
  |
  v
Fabric Gateway SDK
  |
  |-----------------|-----------------|
  v                   v                   v
Peer0 Org1     Peer0 Org2     Orderer
  |-----------------|-----------------|
              v
      Blockchain Ledger
              |
      Certificate Authority
```

Network Management Workflow

Install Dependencies
▼
Start Fabric Network
▼
Create Channel
▼
Deploy Chaincode
▼
Verify Containers
▼
Monitor Logs
▼
Verify Network Status
▼
Stop Network
▼
Clean Resources

Result

The Hyperledger Fabric network was successfully deployed and managed. The blockchain network components, including peer nodes, ordering service, Certificate Authorities, and application channel, were verified. Chaincode was deployed successfully, network status was monitored, and the Fabric network was shut down and cleaned up successfully.